

README + AI_USAGE — LLD Practice Platform

README + AI_USAGE (combined for form upload)

LLD Practice Platform

A focused prototype that lets a learner pick an LLD problem, write a structured text design, submit it, and receive rubric-based feedback (deterministic checks + AI or heuristic judgment), with full attempt history.

> See docs/RESEARCH.md and docs/DESIGN.md for the **why**. Upload-ready copies live under docs/submission/.

Practice loop

Choose problem !' Write design !' Submit !' Evaluate !' Review feedback !' Try again

Tech stack

- API: Node.js + Express + TypeScript
- DB: SQLite via Prisma
- Web: React + Vite + TypeScript
- Evaluation: in-process queue + OpenAiEvaluator or HeuristicEvaluator
- Tests: Vitest

Project structure

apps/api/ Express, domain, Prisma, evaluators, queue
apps/web/ React practice UI
docs/ RESEARCH.md, DESIGN.md
docs/submission/ PDFs / combined upload files
README.md
AI_USAGE.md
.env.example

How to run (development)

From repo root

```
npm install  
cp .env.example apps/api/.env
```

Optional: set OPENAI_API_KEY in apps/api/.env (otherwise heuristic evaluator is used)

EVALUATOR=auto openai heuristic

```
npm run db:setup  
npm run dev:api
```

In another terminal:

```
npm run dev:web
```

- API: <http://localhost:4000> (GET /api/health)
- Web: <http://localhost:5173> (proxies /api to the API)

How to run (production-style single process)

```
npm install  
npm run db:setup  
npm run build  
npm start
```

Express serves the Vite build and the API on <http://localhost:4000>.

How to regenerate submission PDFs

npm run docs:pdf

Writes docs/submission/Research_Note.pdf, Design_Note.pdf, and README_AND_AI_USAGE.pdf (plus the combined .md).

How to run tests

npm test

Coverage includes:

- Attempt state machine and design validation
- Heuristic evaluator fixtures (strong vs thin designs)
- API flow: create !' submit !' evaluate !' history, plus double-submit 409

Key decisions

1. Structured text submission behind a Submission / text_v1 payload — enough evidence for LLD coaching; diagrams/code can be added later.
2. Rubric findings, not a single 0–100 score — each criterion has evidence, concern, suggestion, confidence.
3. Deterministic validation before any evaluator — missing sections fail fast without calling OpenAI.
4. Persist then evaluate — submission is stored before the async worker runs; failures do not lose work.
5. Evaluator port — OpenAiEvaluator and HeuristicEvaluator swap without changing the practice flow.

Known limitations

- Cookie learnerId only — no real auth.
- In-process queue is not durable across server restart.
- Shared rubric across problems (not per-problem tuned).
- AI quality depends on the model/prompt; framed as coaching feedback, not a certified grade.

What I'd build next

1. A second non-LLM rule evaluator to harden the port.
2. A diagram_v1 submission format.
3. Per-problem rubric overrides.
4. Extract the evaluation worker when latency/load requires it.

AI Usage

This document covers meaningful decisions where AI (Claude for planning; Cursor for implementation) was used on the LLD Practice Platform — what was suggested, what was accepted or rejected, and why.

1. Submission format: text vs. diagram vs. code

What AI suggested: Support a combined format — structured text plus an optional lightweight diagram editor — for richer signal from day one.

What I accepted / rejected: Rejected the diagram editor for the MVP; kept text-only structured fields. Accepted the idea that format should be pluggable, which became Submission.format = text_v1 plus a JSON payload (see docs/DESIGN.md, Change Test A).

2. Evaluation prompt design

What AI suggested: An open prompt like “score this design out of 100 with comments.”

What I accepted / rejected: Rejected the open-ended score. Accepted a fixed-rubric JSON shape (criterionId, score, evidence, concern, suggestion, confidence) implemented in OpenAiEvaluator, with evidence required to reference the learner's text.

3. Domain model granularity — separate Attempt from Submission?

What AI suggested: Collapse attempt and submission into one mutable row.

What I accepted / rejected: Rejected the collapse. Kept separate Attempt, Submission, and Evaluation so history preserves each submitted snapshot and its feedback trail.

4. Where to draw the deterministic/AI line

What AI suggested: Let the LLM also check required sections / minimum length “for consistency.”

What I accepted / rejected: Rejected LLM structural checks. Implemented them in `validateDesignForSubmit / attemptLifecycle` before enqueue. Judgment-heavy scoring stays in `OpenAiEvaluator` or `HeuristicEvaluator`.

5. Handling evaluation latency/failure

What AI suggested: A full message broker (retries, DLQ) “to do it properly.”

What I accepted / rejected: Rejected the broker for this prototype. Accepted store-first + async evaluation via `EvaluationQueue` and states submitted !' evaluating !' completed failed, with retry from failed.

6. Cursor implementation: Prisma vs raw SQL

What AI suggested during scaffolding: Either raw better-sqlite3 statements or Prisma.

What I accepted / rejected: Accepted Prisma + SQLite for a clear schema, seed script, and less hand-rolled SQL in a short build. Rejected introducing a second datastore or ORM abstraction layer on top.

7. Cursor implementation: OpenAI-only vs always-on heuristic

What AI suggested: Fail the attempt if `OPENAI_API_KEY` is missing.

What I accepted / rejected: Rejected hard failure. Accepted `createEvaluator()` with `EVALUATOR=auto openai heuristic` so demos and tests work without a key via `HeuristicEvaluator`, while production demos can set the key for richer feedback.